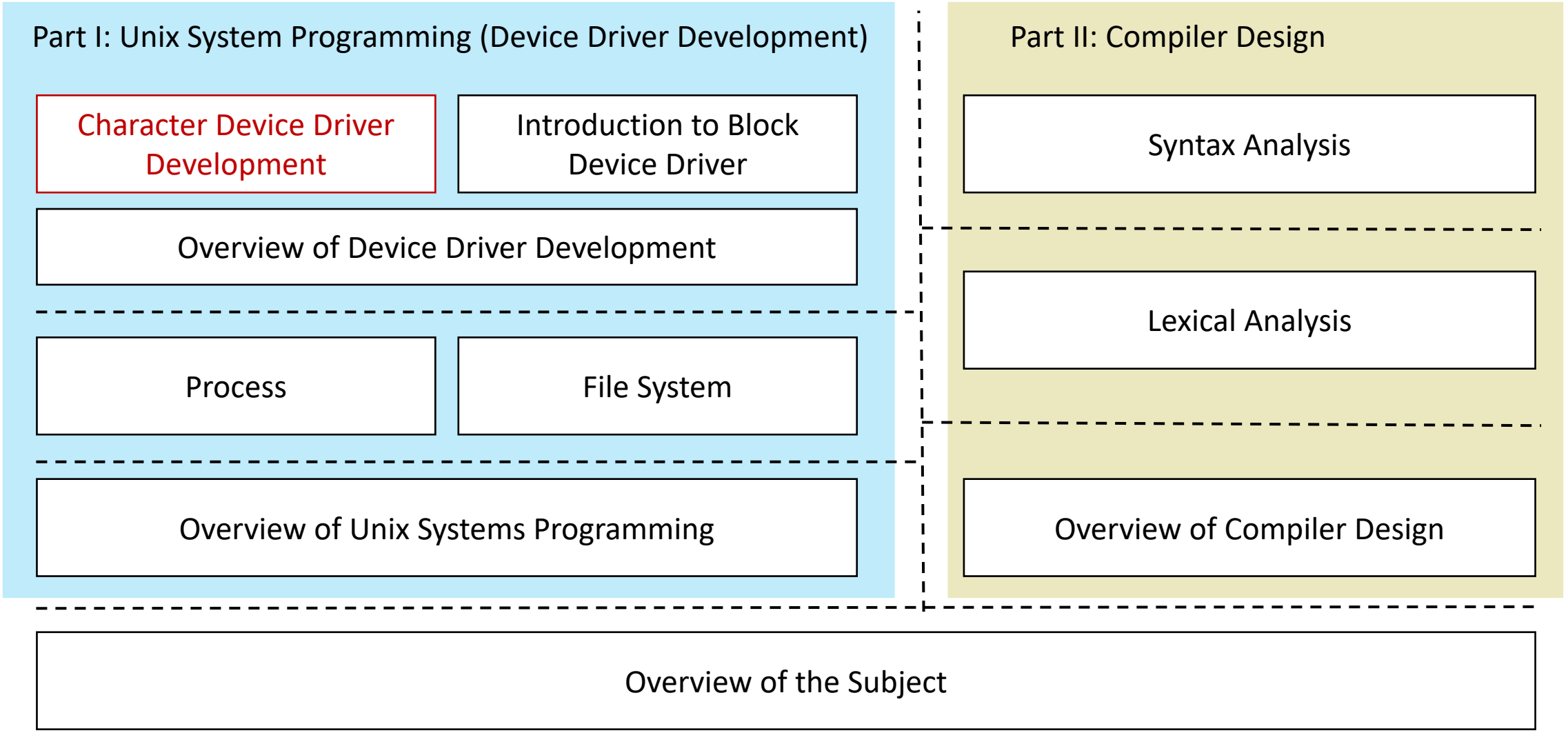
COMP3438 System Programming

Character Device Drivers (I)

Course Organization



Overview of Device Driver Development

In Unix, two generic interfaces are related to character device driver development

User programs: devices are treated as special files and users can only access devices through file operation system call (such as open, close, read, write, etc.), just like accessing regular files.

Kernel: provides a generic interface and kernel routines for all device drivers to implement functionalities and register functions into the kernel data structures (such as char/block device driver tables).

The Interface for User Programs

Device are treated as files: user programs access devices through file operation system calls.

Each file has an inode, and each device driver has a major number.

When using a device driver in user programs, we need to create a special file by associating its major number (driver) with an inode, e.g.

```
mknod /dev/lab1 c 251 0
```

The Device Driver Development (Kernel)

Two tasks

1. Implement functionalities based on the generic interface
2. Register the device driver into the kernel data structure (the char/block device driver table)

The major number is the ID of a device driver

The Generic Interface for Char Device Driver in Linux

A data structure called `file_operations` is defined in `<linux/fs.h>`. It is basically an array of function pointers.

```
struct file_operations{
    struct module * owner;
    …
    int (*open)(struct inode *, struct file *);
    ssize_t (*read)(struct file *, char *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char *, size_t, loff_t *);
    int (*release)(struct inode *, struct file *);
    …
}
```

To develop a char driver, we need to implement the corresponding functions for a device based on the above generic interface.

"Hello World" Device Driver

Demo Task 1

Header files include the prototypes of kernel routines and data structures needed in the program

```
#include <linux/config.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>


#include <linux/fs.h>


#include <linux/string.h>
#include <asm/uaccess.h>
```

Use "static", the effective range of a variable is limited in the file containing it, so we can avoid name pollution.

```
#define DEVICE_NAME "comp309_char_lab1"
static char msg[] = "Hello World!!!\n"
static int major;
```

"Hello World" Device Driver (Continued)

```
static int zili_demo_char1_open(struct inode *inode, struct file *fp){
    MOD_INC_USE_COUNT;
    printk("Device " DEVICE_NAME " open.\n");
    return 0;
}
static ssize_t zili_demo_char1_read(struct file *fp, char *buf, size_t count, loff_t *position){
    int num;
    if (count < strlen(msg)) num = count;
    else num = strlen(msg);
    copy_to_user(buf, msg, num);
    return num;
}
static int zili_demo_char1_release(struct inode *inode, struct file*fp){
    MOD_DEC_USE_COUNT;
    printk("Device " DEVICE_NAME " release.\n");
    return 0;
}
```

"Hello World" Device Driver (Continued)

```
static int zili_demo_char1_open(struct inode *inode, struct file *fp){
    MOD_INC_USE_COUNT;
    printk("Device " DEVICE_NAME " open.\n");
    return 0;
}
static ssize_t zili_demo_ch
    int num;
    if (count < strlen(msg)
    else num = strlen(msg);
    copy_to_user(buf, msg,
    return num;
}
static int zili_demo_char1_
    MOD_DEC_USE_COUNT;
    printk("Device " DEVICE
    return 0;
}
```

Implement functionalities of a device based on the generic interface defined in file_operations.

```
struct file_operations{
    struct module * owner;
    …
    int (*open)(struct inode *, struct file *);
    ssize_t (*read)(struct file *, char *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char *, size_t, loff_t *);
    int (*release)(struct inode *, struct file *);
    …
}
```

"Hello World" Device Driver (Continued)

```
static int zili_demo_char1_open(struct inode *inode, struct file *fp){
    MOD_INC_USE_COUNT;
    printk("Device " DEVICE_NAME " open.\n");
    return 0;
}
static ssize_t zili_demo_char1_read(struct file *fp, char *buf, size_t count, loff_t *position){
    int num;
    if (count < strlen(msg
    else num = strlen(msg)
    copy_to_user(buf, msg,
    return num;
}
static int zili_demo_char1
    MOD_DEC_USE_COUNT;
    printk("Device " DEVIC
    return 0;
}
```

Implement functionalities of a device based on the generic interface defined in file_operations.

```
struct file_operations{
    struct module * owner;
    …
    int (*open)(struct inode *, struct file *);
    ssize_t (*read)(struct file *, char *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char *, size_t, loff_t *);
    int (*release)(struct inode *, struct file *);
    …
}
```

"Hello World" Device Driver (Continued)

```
static int zili_demo_char1_o
    MOD_INC_USE_COUNT;
    printk("Device " DEVICE_
    return 0;
}
static ssize_t zili_demo_cha
    int num;
    if (count < strlen(msg))
    else num = strlen(msg);
    copy_to_user(buf, msg, n
    return num;
}
static int zili_demo_char1_release(struct inode *inode, struct file*fp){
    MOD_DEC_USE_COUNT;
    printk("Device " DEVICE_NAME " release.\n");
    return 0;
}
```

Implement functionalities of a device based on the generic interface defined in file_operations.

```
struct file_operations{
    struct module * owner;
    …
    int (*open)(struct inode *, struct file *);
    ssize_t (*read)(struct file *, char *, size_t, loff_t *);
    ssize_t (*write)(struct file *, const char *, size_t, loff_t *);
    int (*release)(struct inode *, struct file *);
    …
}
```

"Hello World" Device Driver (Continued)

```
static int zili_demo_char1_open(struct inode *inode, struct file *fp){
    MOD_INC_USE_COUNT;
    printk("Device " DEVICE_NAME " open.\n");
    return 0;
}
static ssize_t zili_demo_char1_read(struct file *fp, char *buf, size_t count, loff_t *position){
    int num;
    if (count < strlen(msg)) num = count;
    else num = strlen(msg);
    copy_to_user(buf, msg, num);
    return num;
}
static int zili_demo_char1_release(struct inode *inode, struct file*fp){
    MOD_DEC_USE_COUNT;
    printk("Device " DEVICE_NAME " release.\n");
    return 0;
}
```

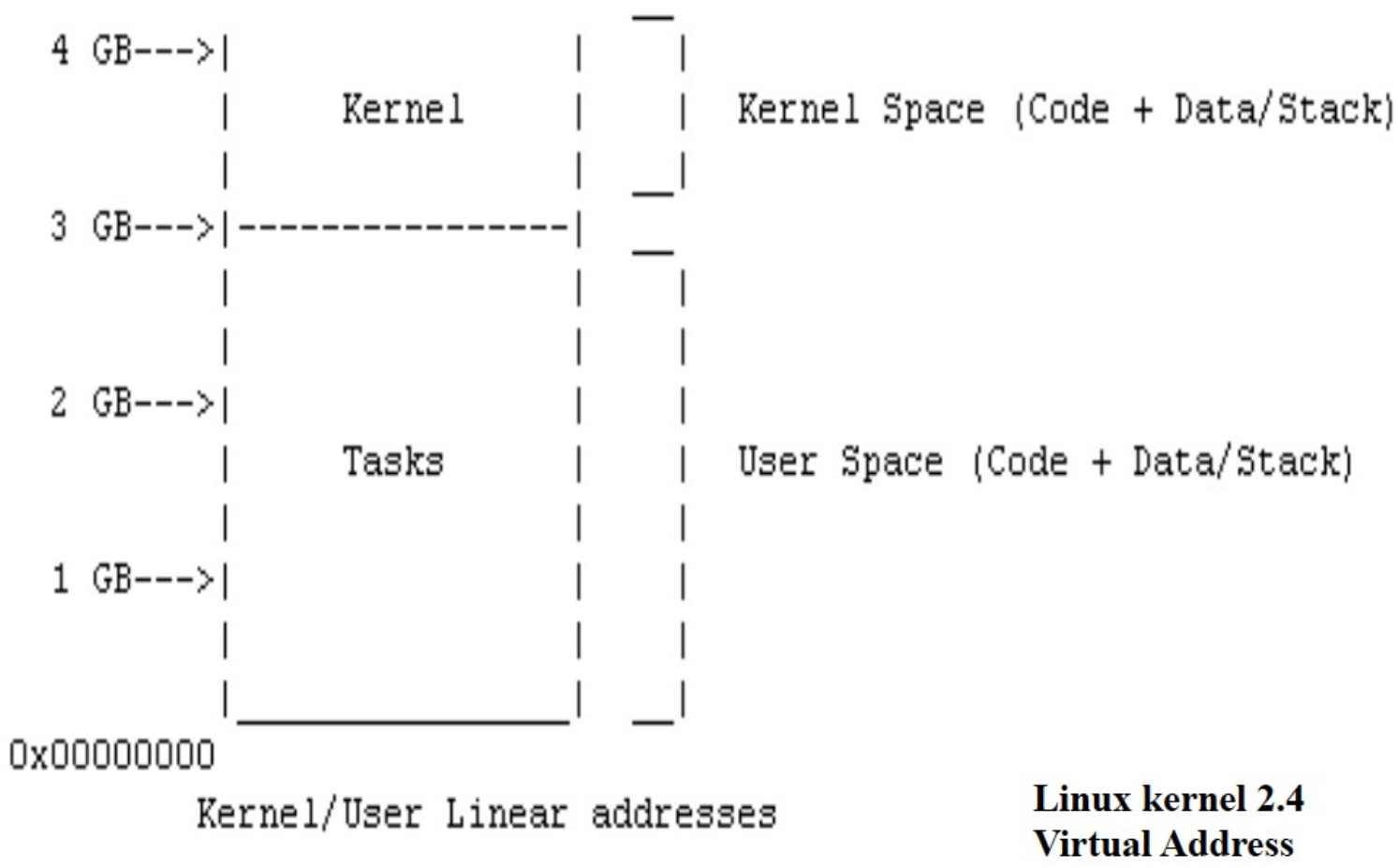
`copy_to_user()` and `copy_from_user()`:

Transferring data between the kernel memory space to/from user memory space.

Kernel Memory Space and User Memory Space

The kernel virtual memory address space and user virtual memory address space are separate in Unix.

The resources that only kernel can access are also called the "kernel land", while the resources that the user can access are also called the "user land".



Linux kernel 2.4 Virtual Address

Register/Unregister a Character Device Driver

Register: inform the kernel that we have a set of generic interface functions implemented for a certain kind of device

```
int register_chrdev(unsigned int major,
    const char * name,
    struct file_operations *fops);
```

`major` – major number. If it is `0`, the system will automatically assign an unused major number, and return it; if it is greater than `0`, this function will attempt to reserve this number as the character device's major number, and return `0` on success.

`name` – a `char` pointer to the name of this character device driver.

`fops` – a pointer to a `file_operations` structure, where pointers to the generic interface functions are stored.

Register/Unregister a Character Device Driver

Unregister: inform the kernel that the specified character device driver should be removed.

```
int unregister_chrdev(unsigned int major,
    const char * name);
```

`major` – major number of the character device.

`name` – a `char` pointer to the name of this character device driver.

"Hello World" Device Driver (Continued)

```
static struct file_operations zili_demo_fops = {
    owner: THIS_MODULE,
    open: zili_demo_char1_open,
    read: zili_demo_char1_read,
    release: zili_demo_char1_release,
};
```

"Hello World" Device Driver (Continued)

```
static struct file_operations zili_demo_fops = {
    owner: THIS_MODULE,
    open: zili_demo_char1_open,
    read: zili_demo_char1_read,
    release: zili_demo_char1_release,
};
```

Used in registering and unregistering

"Hello World" Device Driver (Continued)

```
static int __int zili_demo_char1_init(void){
    int ret;
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialized (major number -- %d).\n", major);
    return 0;
}
static void __exit zili_demo_char1_exit(void){
    unregister_chrdev(major, DEVICE_NAME);
    printk("Device " DEVICE_NAME " unloaded.\n");
}
module_init(zili_demo_char1_init);
module_exit(zili_demo_char1_exit);
```

"Hello World" Device Driver (Continued)

```
static int __int zili_demo_char1_init(void){
    int ret;
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialized (major number -- %d).\n", major);
    return 0;
}
static void __exit zili_demo_char1_exit(void){
    unregister_chrdev(major, DEVICE_NAME);
    printk("Device " DEVICE_NAME " unloaded.\n");
}
module_init(zili_demo_char1_init);
module_exit(zili_demo_char1_exit);
```

Register the character device driver

"Hello World" Device Driver (Continued)

```
static int __int zili_demo_char1_init(void){
    int ret;
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialized (major number -- %d).\n", major);
    return 0;
}
static void __exit zili_demo_char1_exit(void){
    unregister_chrdev(major, DEVICE_NAME);
    printk("Device " DEVICE_NAME " unloaded.\n");
}
module_init(zili_demo_char1_init);
module_exit(zili_demo_char1_exit);
```

Unregister the character device driver

"Hello World" Device Driver (Continued)

```
static int __int zili_demo_char1_init(void){
    int ret;
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialized (major number -- %d).\n", major);
    return 0;
}
static void __exit zili_demo_char1_exit(void){
    unregister_chrdev(major, DEVICE_NAME);
    printk("Device " DEVICE_NAME " unloaded.\n");
}
module_init(zili_demo_char1_init);
module_exit(zili_demo_char1_exit);
```

Rendezvous point with the kernel, on where to start.

"Hello World" Device Driver (Continued)

```
static int __int zili_demo_char1_init(void){
    int ret;
    ret = register_chrdev(0, DEVICE_NAME, &zili_demo_fops);
    if (ret < 0){
        printk("Device " DEVICE_NAME " cannot get major number.\n");
        return ret;
    }
    major = ret;
    printk("Device " DEVICE_NAME " initialized (major number -- %d).\n", major);
    return 0;
}
static void __exit zili_demo_char1_exit(void){
    unregister_chrdev(major, DEVICE_NAME);
    printk("Device " DEVICE_NAME " unloaded.\n");
}
module_init(zili_demo_char1_init);
module_exit(zili_demo_char1_exit);
```

Rendezvous point with the kernel, on where to end.

Compile and Load/Unload

Linux provides a dynamic module load/unload method for device driver management.

Compile:

Add module dependency relations into `Makefile` and `KConfig`

Run command "`make menuconfig`" to set up the module as "`dynamically load/unload`"

Run command "`make`"

Load/Unload:

```
insmod xxx.ko
rmmod xxx
```

where xxx is the module name.

Test

To the user land, devices are treated as files so we need to create a file for a device driver using command "`mknod`", e.g.

```
mknod /dev/lab1 c 251 0
```

User programs access devices through file operation system calls, e.g.

```
int fd;
fd = open("/dev/lab1", O_RDONLY);
```

Application Program

```
#include <linux/types.h>
#include <linux/stat.h>
#include <linux/fcntl.h>
main(){
    int fd, count, I;
    char buf[100];
    fd = open("/dev/lab1", O_RDONLY);
    count = read(fd, buf, 100);
    printf("count-%d\n", count);
    buf[count] = '\0';
    printf("%s", buf);
}
```

Summary

Two generic interfaces are provided for using devices in Unix:

- In the kernel, a generic interface is provided.
- For user programs, devices are treated as files.

To develop a device driver, basically we need to finish two tasks:

- Implement functionalities based on the generic interface
- Register the device driver into the kernel data structure (the char/block device driver table)

A simple device driver "Hello" has been provided as an example for understanding the internal of a character devcie driver.